SPECIFICATION DOC V2.0

romx & romx-pack Workspace Specification

Complete Technical Architecture, CLI Tools, Format Conversion Pipeline, & Determinism Engine

1. Executive Summary

The `romx` ecosystem consists of a core Rust library crate (`romx`) and a command-line interface tool (`romx-pack`). It establishes an immutable, single-file container format (`.romx`) designed to combine retro ROMs, tape/disk media, dependencies, preview media, cheats, and historical metadata.

Key capabilities include zero-copy memory mapping (`mmap`) via 4KB boundary alignment, multi-mode set packaging (Non-Merged, Merged, Split), automatic format conversion (ZIP, CHD, Spectrum TZX/TAP), and TorrentZip-inspired deterministic byte compilation for peer-to-peer distribution.

2. Binary Container Layout (.romx)

```
+-------------------------------------------------------------------------------+
| 1. Header Block (32 Bytes Fixed, repr(C, packed))                             |
|    - magic: "ROMX" (0x52 0x4F 0x4D 0x58)                                      |
|    - version: u16 (0x0001)                                                    |
|    - flags: u16 (Bit 0 = Compressed Assets, Bit 1 = Read Only)                |
|    - sqlite_offset: u64 (Default: 32)                                         |
|    - sqlite_size: u64                                                         |
|    - payload_offset: u64 (Aligned to 4096-byte sector boundary)               |
+-------------------------------------------------------------------------------+
| 2. Embedded Read-Only SQLite Database                                         |
|    - PRAGMA page_size = 4096                                                  |
|    - Indexed assets, manifests, game metadata, cheats, and tape block markers |
|    - Padded to nearest 4096-byte boundary                                     |
+-------------------------------------------------------------------------------+
| 3. Contiguous Binary Payload Block                                            |
|    - Uncompressed ROM streams aligned to 4096-byte boundaries (Zero-copy mmap) |
|    - Compressed UI Media Assets (zlib / zstd) & embedded CHD/tape payloads    |
+-------------------------------------------------------------------------------+
```

3. Embedded SQLite Schema

```
CREATE TABLE manifest (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

CREATE TABLE assets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    virtual_path TEXT UNIQUE NOT NULL,
    hash_sha256 TEXT NOT NULL,
    storage_type INTEGER NOT NULL,       -- 0 = EMBEDDED, 1 = EXTERNAL_REFERENCE
    compression_algo INTEGER DEFAULT 0,  -- 0 = None (mmap ready), 1 = zlib, 2 = zstd
    uncompressed_size INTEGER NOT NULL,
    compressed_size INTEGER NOT NULL,
    payload_offset INTEGER,
    alignment_bytes INTEGER DEFAULT 0
);

CREATE TABLE tape_blocks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    block_index INTEGER NOT NULL,
    block_type TEXT NOT NULL,
    description TEXT,
    payload_offset INTEGER NOT NULL
);
```

4. Command-Line Syntax & Conversion Pipeline

4.1 Subcommands Summary

Command	Arguments / Flags	Description
`build`	`--mode, --dat, --rom-path, --out`	Assembles Non-Merged, Merged, or Split containers from DAT/ROM sets.
`convert`	`--input, --chd, --media, --out`	Converts ZIP, MAME CHD, ZX Spectrum TAP/TZX files into `.romx` format.
`inspect`	`<file.romx>`	Outputs JSON breakdown of offsets, assets, and mmap readiness.
`verify`	`<file.romx>`	Re-computes payload SHA-256 hashes against SQLite records.
`extract`	`<file.romx> --out-dir <path>`	Unpacks raw ROMs, SQLite DB, media, and cheats to directory tree.

4.2 Ingestion & Conversion Drivers

- **ZipConverter:** Iterates ZIP entries, computes ***SHA-256***, and maps files to POSIX paths.
- **ChdConverter:** Parses CHD hunk structures and embeds raw streams with `compression_algo = 0` to prevent double-compression penalties.
- **TapeConverter:** Reads Spectrum TAP/TZX pulse blocks, embedding binary payloads while generating index rows in `tape_blocks` for fast UI block-jumping.

5. TorrentZip-Inspired Determinism Rules

> **Guaranteed Byte Hashing Compliance**
>
> To ensure identical container outputs regardless of host operating system or execution date:

1. **Path Normalization:** All separators are converted to POSIX forward slashes (`/`).
2. **Canonical Sorting:** `virtual_path` entries are sorted in lowercase ASCII order prior to writing database tables or binary payloads.
3. **System Metadata Stripping:** File modification dates (`mtime`), attributes, and host OS permissions are stripped from SQLite records.
4. **Fixed Encoder Configuration:** Compression libraries (`zstd` / `zlib`) enforce fixed compression levels to prevent encoder output variations.